

Design Patterns in C# - From Fundamentals to Real-World Use

1) What Are Design Patterns?

Design patterns are proven, reusable solutions to recurring design problems in software. They are not copy-paste code and not framework-specific APIs. A pattern is:

- a **named approach** to a common problem,
- with a known **intent**,
- with known **trade-offs**,
- and a common **vocabulary** teams can share.

When someone says, "let's use Strategy here," they compress a lot of design meaning into one phrase.

Why Patterns Exist

Patterns exist because software systems repeatedly face similar pressures:

- changing requirements,
- need for extension without breaking code,
- object creation complexity,
- runtime behavior variation,
- reducing coupling across modules.

Patterns help balance flexibility, readability, and maintainability.

2) A Brief History

The modern software pattern movement was inspired by Christopher Alexander (1936-2022), an architect and design theorist.

Alexander was not a programmer. He worked in building and urban design.

His key idea was that successful places (rooms, streets, neighborhoods) repeatedly use certain proven design solutions to recurring human problems.

He called these reusable solutions **patterns**, and he introduced the idea of a **pattern language**:

- each pattern has a context and a recurring problem,
- each pattern proposes a solution,
- patterns connect to each other as a system, not as isolated tricks.

In simple terms, he gave designers a vocabulary for solving recurring design problems consistently.

Software engineers recognized that codebases have the same kind of recurring design problems:

- object creation responsibilities,
- coupling between modules,
- changing behavior over time.

So the architecture idea was transferred into software design:

- identify recurring design problems,
- document proven object-oriented solutions,
- name them so teams can discuss design quickly and precisely.

In software:

- **1987-1992**: Early pattern communities and conferences formed.
- **1994**: "Design Patterns: Elements of Reusable Object-Oriented Software" by Gamma, Helm, Johnson, Vlissides (GoF) popularized 23 patterns.
- Since then: patterns expanded and adapted for enterprise systems, distributed systems, cloud, and domain-driven design.

Connection in one sentence:

Alexander provided the methodology ("pattern language" for recurring design problems), and software engineering applied that methodology to object-oriented program design.

Important: patterns are not laws. They are tools.

Expert engineers choose patterns only when they solve a real problem.

3) Pattern Classification (Why 3 Groups?)

Classic GoF patterns are grouped by **intent**, not complexity.

1. **Creational** - how objects are created.
2. **Structural** - how objects/classes are composed.
3. **Behavioral** - how objects communicate and share responsibilities.

Classification Map

Design Patterns

```
|  
+-- Creational  -> "How do we create objects safely/flexibly?"  
|  
+-- Structural  -> "How do we assemble objects into larger structures?"  
|  
+-- Behavioral  -> "How do objects collaborate and vary behavior?"
```

This grouping matters because design problems typically start in one of these three dimensions.

4) Anti-Patterns: What They Are and Why They Matter

An anti-pattern is a commonly seen "solution" that looks useful at first but causes long-term harm.

Design patterns capture **good recurring solutions**.

Anti-patterns capture **bad recurring solutions**.

Common Anti-Patterns (with quick meaning)

1. **God Object / Blob**
 - One class knows and does too much.
 - Symptoms: huge class, many dependencies, hard to test.
2. **Spaghetti Code**
 - Control flow is tangled and hard to follow.
 - Symptoms: deep nesting, inconsistent structure, fear of change.
3. **Copy-Paste Programming**
 - Duplicating logic instead of abstraction.

- Symptoms: bug fixes repeated in many places.

4. **Lava Flow**

- Dead/obsolete code remains because nobody dares remove it.
- Symptoms: mysterious modules, unclear ownership.

5. **Golden Hammer**

- Using one preferred tool/pattern for every problem.
- Symptoms: forced architecture, overengineering.

6. **Singleton Overuse (important)**

- Treating global mutable state as a convenience.
- Symptoms: hidden coupling, order-dependent tests, concurrency risks.

Anti-Pattern vs Pattern Thinking

Pattern mindset: "What problem is recurring, and what trade-off is acceptable?"

Anti-pattern mindset: "What is fastest now, regardless of long-term cost?"

5) Catalog of Patterns by Classification

Below is a compact catalog of the most widely used classic patterns.

Creational

- Singleton
- Factory Method
- Abstract Factory
- Builder
- Prototype

Structural

- Adapter
- Decorator
- Composite
- Facade
- Proxy
- Bridge

- Flyweight

Behavioral

- Command
- Iterator
- Strategy
- State
- Observer
- Chain of Responsibility
- Template Method
- Mediator
- Visitor
- Memento

The sections below focus on the patterns most useful for day-to-day C# architecture.

6) Learning Order

The order below is chosen for understanding progression:

1. Singleton
2. Factory Method
3. Abstract Factory
4. Builder
5. Adapter
6. Decorator
7. Composite
8. Iterator
9. Observer
10. Command
11. Strategy
12. State
13. Chain of Responsibility

Why this order:

- Start with object creation fundamentals.

- Move to structure/composition.
- Finish with behavior orchestration and runtime variation.

7) Creational Patterns

7.1 Singleton

Intent

Ensure a class has only one instance and provide a global access point to it.

Why It Is Creational

Singleton controls **how and when instance creation happens**.

Therefore, it belongs to creational patterns.

Why Singleton Is Also Called an Anti-Pattern (Sometimes)

Singleton is often criticized because it can become hidden global state.

Risks:

- implicit dependencies (harder to reason about),
- harder unit testing,
- shared mutable state across threads,
- tight coupling to concrete implementation.

Use Singleton mainly for stateless services or immutable shared components.

Avoid it for business state.

Visualization

```
Clients ----> [ AppConfig.Instance ] ----> same shared object
      |               ^
      +-----+
```

C# Examples (Lazy and Non-Lazy)

Lazy Singleton:

```
public sealed class AppConfig
{
    private static readonly Lazy<AppConfig> _instance =
        new(() => new AppConfig());

    public static AppConfig Instance => _instance.Value;

    public string Environment { get; } = "Production";

    private AppConfig() { }
}
```

Non-lazy (eager initialization) Singleton:

```
public sealed class AppConfigEager
{
    private static readonly AppConfigEager _instance = new();

    public static AppConfigEager Instance => _instance;

    public string Environment { get; } = "Production";

    private AppConfigEager() { }
}
```

Usage

- configuration snapshot,
- shared cache metadata (carefully),
- central immutable settings provider.

Examples

- ILoggerFactory -like central provider that returns one shared logging configuration source,
- application-wide immutable feature-flag snapshot loaded once at startup.

Edge Cases

- accidental hidden global state can leak between tests and request scopes,
- mutable singleton fields can create race conditions in multithreaded apps,
- initialization that depends on external resources (file system/network) can fail at startup.

Best Practices

- keep singleton instances stateless or immutable whenever possible,
- expose behavior through interfaces and register via DI for testability,
- avoid storing per-user, per-request, or transaction state in singletons.

7.2 Factory Method

Intent

Define an interface for creating an object, but let subclasses decide which class to instantiate.

Problem It Solves

When code should depend on abstractions, but concrete type selection varies by context.

Visualization

```
Creator (abstract)
|
+-- Create() -> Product
    ^
    |
ConcreteCreatorA -> ConcreteProductA
ConcreteCreatorB -> ConcreteProductB
```


C# Example

```
public interface IReport
{
    string Render();
}

public class PdfReport : IReport
{
    public string Render() => "PDF report";
}

public class ExcelReport : IReport
{
    public string Render() => "Excel report";
}

public abstract class ReportFactory
{
    public abstract IReport Create();
}

public class PdfReportFactory : ReportFactory
{
    public override IReport Create() => new PdfReport();
}

public class ExcelReportFactory : ReportFactory
{
    public override IReport Create() => new ExcelReport();
}
```

Usage

- exporter selection by format,
- storage provider creation by environment,
- pluggable integrations.

Examples

- report generation module choosing PdfReport or ExcelReport based on requested output format,

- notification sender choosing a provider-specific sender implementation (SES, SendGrid, SMTP).

Edge Cases

- subclass count can grow quickly when product variants multiply,
- clients can accidentally depend on concrete creators, defeating abstraction,
- factory methods that require many parameters often indicate missing parameter object/value object design.

Best Practices

- return interfaces or abstract product types, never concrete internals,
- keep creator classes focused on creation logic, not business logic,
- combine with DI/container registration so selection logic is centralized.

7.3 Abstract Factory

Intent

Provide an interface for creating families of related objects without specifying their concrete classes.

Problem It Solves

You need multiple related objects that must stay compatible (e.g., UI theme families, cloud provider families).

Visualization

```
AbstractFactory
```

```
  +-- CreateButton() -> IButton
```

```
  +-- CreateDialog() -> IDialog
```

```
WinFactory ----> WinButton, WinDialog
```

```
MacFactory ----> MacButton, MacDialog
```

C# Example

```
public interface IButton { string Paint(); }
public interface IDialog { string Show(); }

public class WinButton : IButton { public string Paint() => "Windows Button"; }
public class WinDialog : IDialog { public string Show() => "Windows Dialog"; }

public class MacButton : IButton { public string Paint() => "Mac Button"; }
public class MacDialog : IDialog { public string Show() => "Mac Dialog"; }

public interface IUiFactory
{
    IButton CreateButton();
    IDialog CreateDialog();
}

public class WinUiFactory : IUiFactory
{
    public IButton CreateButton() => new WinButton();
    public IDialog CreateDialog() => new WinDialog();
}

public class MacUiFactory : IUiFactory
{
    public IButton CreateButton() => new MacButton();
    public IDialog CreateDialog() => new MacDialog();
}
```

Key Difference from Factory Method

- Factory Method creates **one product type**.
- Abstract Factory creates **a family of related product types**.

Usage

- cross-platform UI kits (Windows/Mac/Linux families),
- environment bundles (dev/staging/prod implementations),
- cloud provider families (storage, queue, notifications with vendor consistency).

Examples

- cloud deployment package that creates compatible `S3Storage` + `SqsQueue` + `SesNotifier` family for AWS,
- UI toolkit that creates `MacButton` + `MacDialog` + `MacMenu` together to keep look and behavior consistent.

Edge Cases

- adding a new product type (new method in factory) forces changes in all concrete factories,
- family mismatch can happen if objects from different factories are mixed accidentally,
- too many factory layers can make simple code harder to follow.

Best Practices

- use Abstract Factory only when product families must remain compatible,
- keep each factory family cohesive and clearly named,
- prefer composition over inheritance inside concrete products.

7.4 Builder

Intent

Separate construction of a complex object from its representation so the same construction process can create different representations.

Problem It Solves

When constructors become unreadable due to many optional parameters and creation rules.

Visualization

Director (optional)

|

v

Builder -> step1 -> step2 -> step3 -> Build() -> Product

C# Example (Fluent Builder)

```
public class Invoice
{
    public string Customer { get; set; } = "";
    public List<string> Lines { get; set; } = new();
    public decimal Discount { get; set; }
}

public class InvoiceBuilder
{
    private readonly Invoice _invoice = new();

    public InvoiceBuilder ForCustomer(string name)
    {
        _invoice.Customer = name;
        return this;
    }

    public InvoiceBuilder AddLine(string line)
    {
        _invoice.Lines.Add(line);
        return this;
    }

    public InvoiceBuilder WithDiscount(decimal discount)
    {
        _invoice.Discount = discount;
        return this;
    }

    public Invoice Build() => _invoice;
}
```

Note: In production C#, prefer immutable objects and builders that collect state separately before creating the final immutable object.

Usage

- constructing domain objects with many optional fields,
- creating test data builders for readable unit tests,
- assembling configuration payloads step-by-step with validation.

Examples

- `OrderBuilder` used in tests to quickly produce valid orders with only the fields needed per scenario,
- query builder composing optional filters (date range, status, region) before creating final request DTO.

Edge Cases

- builders can accidentally expose partially built invalid objects,
- reusing the same builder instance across multiple requests can leak state,
- very large fluent APIs can become ambiguous and hard to maintain.

Best Practices

- validate required fields in `Build()` and fail fast with clear errors,
- treat builders as short-lived objects; create a new builder per product,
- prefer immutable final products and keep mutation inside builder only.

8) Structural Patterns

8.1 Adapter

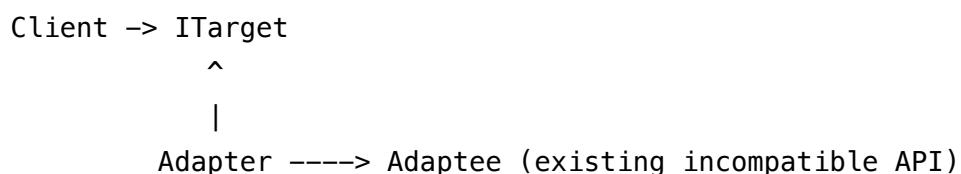
Intent

Convert the interface of a class into another interface clients expect.

Problem It Solves

Integrating legacy or third-party code without changing its source.

Visualization



C# Example

```
public interface IPaymentGateway
{
    PaymentResult Charge(PaymentRequest request);
}

public class PaymentRequest
{
    public decimal Amount { get; set; }
    public string Currency { get; set; } = "";
    public string CardToken { get; set; } = "";
}

public class PaymentResult
{
    public bool Success { get; set; }
    public string? ProviderTransactionId { get; set; }
    public string? ErrorMessage { get; set; }
}

// Third-party SDK we cannot modify.
public class ThirdPartyPaySdk
{
    public SdkChargeResponse ExecuteCharge(SdkChargeRequest request)
    {
        // Real SDK call happens here.
        return new SdkChargeResponse
        {
            Status = "OK",
            TxId = Guid.NewGuid().ToString("N"),
            Error = null
        };
    }
}

public class SdkChargeRequest
{
    public long AmountInCents { get; set; }
    public string CurrencyCode { get; set; } = "";
    public string Token { get; set; } = "";
}
```

```

public class SdkChargeResponse
{
    public string Status { get; set; } = "";
    public string? TxId { get; set; }
    public string? Error { get; set; }
}

public class PaymentGatewayAdapter : IPaymentGateway
{
    private readonly ThirdPartyPaySdk _sdk;

    public PaymentGatewayAdapter(ThirdPartyPaySdk sdk) => _sdk = sdk;

    public PaymentResult Charge(PaymentRequest request)
    {
        var sdkRequest = new SdkChargeRequest
        {
            AmountInCents = (long)(request.Amount * 100m),
            CurrencyCode = request.Currency,
            Token = request.CardToken
        };

        var sdkResponse = _sdk.ExecuteCharge(sdkRequest);

        // Adapter translates provider response into our internal contract.
        var success = string.Equals(sdkResponse.Status, "OK", StringComparison.Ordinal)
        return success
            ? new PaymentResult
            {
                Success = true,
                ProviderTransactionId = sdkResponse.TxId,
                ErrorMessage = null
            }
            : new PaymentResult
            {
                Success = false,
                ProviderTransactionId = null,
                ErrorMessage = sdkResponse.Error ?? "Provider rejected payment"
            };
    }
}

```


Usage

- wrapping legacy services to match modern interfaces,
- integrating third-party SDKs without changing application contracts,
- migrating old modules incrementally behind a stable adapter boundary.

Examples

- checkout service depends only on `IPaymentGateway` , while adapter translates to an external card processor SDK,
- shipping module wraps a courier REST client and maps provider statuses to internal `ShipmentStatus` ,
- accounting module adapts a bank file parser (fixed-width format) to a clean `IStatementImporter` interface.

Edge Cases

- semantic mismatch: converted methods may look compatible but behave differently,
- adapter chains can become deep and hurt observability/debugging,
- error models may differ (exception-based vs return-code based) and require translation.

Best Practices

- keep adapters thin; translate only interface concerns, not business rules,
- document behavioral differences clearly when adaptation is lossy,
- add contract tests ensuring adapter behavior matches target interface expectations.

8.2 Decorator

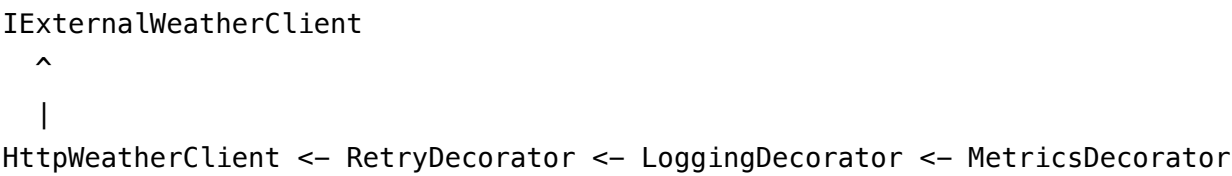
Intent

Attach additional responsibilities to an object dynamically.

Problem It Solves

Adding combinations of features without creating subclass explosion.

Visualization



C# Example

```
public interface IExternalWeatherClient
{
    string GetCurrent(string city);
}

public class HttpWeatherClient : IExternalWeatherClient
{
    public string GetCurrent(string city)
    {
        return $"{city}: 29C, Sunny";
    }
}

public abstract class WeatherClientDecorator : IExternalWeatherClient
{
    protected readonly IExternalWeatherClient Inner;

    protected WeatherClientDecorator(IExternalWeatherClient inner) => Inner = inner;

    public virtual string GetCurrent(string city) => Inner.GetCurrent(city);
}

public class LoggingWeatherClientDecorator : WeatherClientDecorator
{
    public LoggingWeatherClientDecorator(IExternalWeatherClient inner) : base(inner) {

    public override string GetCurrent(string city)
    {
        Console.WriteLine($"[LOG] Requesting weather for {city}");
        var result = Inner.GetCurrent(city);
        Console.WriteLine($"[LOG] Response: {result}");
        return result;
    }
}

// Composition:
// IExternalWeatherClient client =
//     new LoggingWeatherClientDecorator(new HttpWeatherClient());
```

Usage

- adding logging, caching, retry, metrics, compression, encryption around core behavior,
- building configurable middleware-like pipelines at runtime,
- enabling feature combinations without subclass explosion.

Examples

- weather API client decorated with retry + logging + metrics for resiliency and observability,
- payment provider client wrapped with idempotency + audit + tracing decorators in production,
- file storage client decorated with encryption + compression + access logging.

Edge Cases

- decorator order matters and can change behavior subtly,
- too many nested decorators make stack traces and debugging harder,
- duplicated side effects (e.g., repeated logging) can happen if wrapping is misconfigured.

Best Practices

- keep decorators single-responsibility and orthogonal,
- define and document expected composition order,
- expose factory/composition helpers to assemble common decorator stacks safely.

8.3 Composite

Intent

Compose objects into tree structures to represent part-whole hierarchies.

Composite lets clients treat individual objects and compositions uniformly.

Problem It Solves

Working with recursive object trees (files/folders, menu items, organization charts).

Visualization

```
CatalogNode
|-- ProductItem (leaf)
|-- CategoryNode (composite)
    |-- ProductItem
    |-- ProductItem
    |-- CategoryNode
```

C# Example

```
public interface ICatalogNode
{
    decimal GetTotalPrice();
    void Print(string indent = "");
}

public class ProductItem : ICatalogNode
{
    public string Name { get; }
    public decimal Price { get; }

    public ProductItem(string name, decimal price)
    {
        Name = name;
        Price = price;
    }

    public decimal GetTotalPrice() => Price;

    public void Print(string indent = "") =>
        Console.WriteLine($"{indent}- Product: {Name} (${Price})");
}

public class CategoryNode : ICatalogNode
{
    private readonly List<ICatalogNode> _children = new();
    public string Name { get; }

    public CategoryNode(string name) => Name = name;

    public void Add(ICatalogNode node) => _children.Add(node);

    public decimal GetTotalPrice() => _children.Sum(x => x.GetTotalPrice());

    public void Print(string indent = "")
    {
        Console.WriteLine($"{indent}+ Category: {Name}");
        foreach (var child in _children)
            child.Print(indent + " ");
    }
}
```

```
// Example usage:
var phones = new CategoryNode("Phones");
phones.Add(new ProductItem("iPhone 15", 999m));
phones.Add(new ProductItem("Galaxy S24", 899m));

var accessories = new CategoryNode("Accessories");
accessories.Add(new ProductItem("Charger", 39m));
phones.Add(accessories);

phones.Print();
Console.WriteLine($"Total price: {phones.GetTotalPrice()}");
```

Another Example (Different Purpose)

```
public interface IPermissionNode
{
    bool CanAccess(string permission);
}

public class PermissionLeaf : IPermissionNode
{
    private readonly HashSet<string> _permissions;

    public PermissionLeaf(params string[] permissions) =>
        _permissions = new HashSet<string>(permissions, StringComparer.OrdinalIgnoreCase);

    public bool CanAccess(string permission) => _permissions.Contains(permission);
}

public class PermissionGroup : IPermissionNode
{
    private readonly List<IPermissionNode> _children = new();
    public void Add(IPermissionNode node) => _children.Add(node);
    public bool CanAccess(string permission) => _children.Any(x => x.CanAccess(permission));
}

// Usage:
var adminRole = new PermissionGroup();
adminRole.Add(new PermissionLeaf("users.read", "users.write"));
adminRole.Add(new PermissionLeaf("billing.read"));

var supportRole = new PermissionGroup();
supportRole.Add(new PermissionLeaf("tickets.read", "tickets.write"));

var platformTeam = new PermissionGroup();
platformTeam.Add(adminRole);
platformTeam.Add(supportRole);

Console.WriteLine(platformTeam.CanAccess("users.write")); // True
Console.WriteLine(platformTeam.CanAccess("billing.write")); // False
```

Usage

- file systems, UI component trees, menu hierarchies, organization structures,

- aggregate operations such as total size, recursive search, or rendering,
- treating leaf and container nodes uniformly in client code.

Examples

- permissions tree where access check runs uniformly on folders and individual files,
- ecommerce category tree where each node can compute combined product count recursively,
- role/permission groups where effective access is resolved recursively across nested permission sets.

Edge Cases

- deep trees can cause recursion depth/performance problems,
- cycles (accidental parent-child loops) can create infinite traversal,
- operations may differ between leaf and composite and require sensible defaults.

Best Practices

- protect tree integrity (prevent cycles, invalid parent assignment),
- provide iterative traversal for very deep trees when needed,
- define clear behavior contracts for unsupported operations on leaf nodes.

9) Behavioral Patterns

9.1 Iterator

Intent

Provide a way to access elements of an aggregate object sequentially without exposing its underlying representation.

Problem It Solves

Traversal logic should be separate from collection internals.

In C#, `IEnumerable<T>` / `IEnumerator<T>` is the built-in iterator model.

C# Example

```
public class EvenNumbers : IEnumerable<int>
{
    private readonly int _max;
    public EvenNumbers(int max) => _max = max;

    public IEnumerator<int> GetEnumerator()
    {
        for (int i = 0; i <= _max; i += 2)
            yield return i;
    }

    IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
}
```

Usage

- custom collections and domain-specific sequences,
- streaming data processing with deferred execution,
- exposing read-only traversal without leaking internal representation.

Examples

- paginated API results exposed as `IEnumerable<T>` to process records lazily,
- custom graph traversal iterator returning BFS/DFS sequence without exposing graph internals.

Edge Cases

- deferred execution can surprise consumers if source data mutates later,
- multiple enumeration may re-run expensive iteration logic,
- iterator side effects can produce inconsistent results across enumerations.

Best Practices

- keep iterators side-effect free and deterministic,
- document enumeration cost and whether multiple passes are safe,
- materialize (`ToList`) when snapshot behavior is required.

9.2 Observer

Intent

Define a one-to-many dependency so when one object changes state, all dependents are notified automatically.

Problem It Solves

Decoupled event-driven updates.

Visualization

```
Subject (Publisher)
| notify
+--> Observer A
+--> Observer B
+--> Observer C
```

C# Example

```
public interface IObserver
{
    void Update(string value);
}

public class PriceFeed
{
    private readonly List<IObserver> _observers = new();
    private string _price = "";

    public void Subscribe(IObserver observer) => _observers.Add(observer);
    public void Unsubscribe(IObserver observer) => _observers.Remove(observer);

    public void SetPrice(string price)
    {
        _price = price;
        foreach (var observer in _observers.ToList())
            observer.Update(_price);
    }
}

public class PriceDashboard : IObserver
{
    public void Update(string value) =>
        Console.WriteLine($"[Dashboard] New price: {value}");
}

public class PriceAlertService : IObserver
{
    public void Update(string value)
    {
        if (decimal.TryParse(value, out var price) && price > 150m)
            Console.WriteLine($"[Alert] Price crossed threshold: {price}");
    }
}

// Usage:
var feed = new PriceFeed();
var dashboard = new PriceDashboard();
var alerts = new PriceAlertService();
```

```
feed.Subscribe(dashboard);  
feed.Subscribe(alerts);  
  
feed.SetPrice("120"); // dashboard + alert service receive update  
feed.SetPrice("170"); // dashboard + threshold alert  
  
feed.Unsubscribe(alerts);  
feed.SetPrice("180"); // only dashboard receives update
```

In .NET, events and `IObservable<T>` are common Observer implementations.

Usage

- domain events, price feeds, notifications, UI/event-driven updates,
- decoupling publishers from unknown or evolving subscriber sets,
- plugin systems where subscribers are loaded dynamically,
- audit and monitoring listeners that can be attached/removed at runtime.

Examples

- order service publishes `OrderPlaced` event consumed by billing, inventory, and email modules,
- stock ticker feed pushes price updates to dashboard, alert engine, and persistence service.

Edge Cases

- memory leaks occur when subscribers are not unsubscribed,
- event storms can overwhelm slow subscribers,
- publisher exceptions can break notification chains if not isolated.

Best Practices

- provide explicit unsubscribe/disposal mechanics,
- isolate subscriber failures so one observer does not break others,
- consider event queueing and backpressure for high-volume notifications.

9.3 Command

Intent

Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue/log requests, and support undo.

Problem It Solves

Decoupling invoker from receiver; enabling replay, scheduling, undo/redo.

Visualization

```
Invoker ----> ICommand.Execute()  
              |  
              v  
            Receiver.Action()
```

C# Example

```
public interface ICommand
{
    void Execute();
}

public class OrderService
{
    public void CreateOrder(string orderId) =>
        Console.WriteLine($"Order created: {orderId}");

    public void CancelOrder(string orderId) =>
        Console.WriteLine($"Order canceled: {orderId}");
}

public class CreateOrderCommand : ICommand
{
    private readonly OrderService _service;
    private readonly string _orderId;

    public CreateOrderCommand(OrderService service, string orderId)
    {
        _service = service;
        _orderId = orderId;
    }

    public void Execute() => _service.CreateOrder(_orderId);
}

public class CancelOrderCommand : ICommand
{
    private readonly OrderService _service;
    private readonly string _orderId;

    public CancelOrderCommand(OrderService service, string orderId)
    {
        _service = service;
        _orderId = orderId;
    }

    public void Execute() => _service.CancelOrder(_orderId);
}
```

```
public class CommandQueue
{
    private readonly Queue<ICommand> _commands = new();

    public void Enqueue(ICommand command) => _commands.Enqueue(command);

    public void ProcessAll()
    {
        while (_commands.Count > 0)
            _commands.Dequeue().Execute();
    }
}

// Usage:
var service = new OrderService();
var queue = new CommandQueue();
queue.Enqueue(new CreateOrderCommand(service, "ORD-1001"));
queue.Enqueue(new CancelOrderCommand(service, "ORD-1002"));
queue.ProcessAll();
```


Undo/Redo Extension

```
public interface IReversibleCommand
{
    void Execute();
    void Undo();
}

public class TextEditor
{
    public string Content { get; private set; } = "";
    public void Append(string text) => Content += text;
    public void RemoveLast(int count) =>
        Content = count >= Content.Length ? "" : Content.Substring(0, Content.Length - count);
}

public class AppendTextCommand : IReversibleCommand
{
    private readonly TextEditor _editor;
    private readonly string _text;

    public AppendTextCommand(TextEditor editor, string text)
    {
        _editor = editor;
        _text = text;
    }

    public void Execute() => _editor.Append(_text);
    public void Undo() => _editor.RemoveLast(_text.Length);
}

public class HistoryManager
{
    private readonly Stack<IReversibleCommand> _undoStack = new();
    private readonly Stack<IReversibleCommand> _redoStack = new();

    public void Execute(IReversibleCommand command)
    {
        command.Execute();
        _undoStack.Push(command);
        _redoStack.Clear();
    }
}
```

```

public void Undo()
{
    if (_undoStack.Count == 0) return;

    var command = _undoStack.Pop();
    command.Undo();
    _redoStack.Push(command);
}

public void Redo()
{
    if (_redoStack.Count == 0) return;

    var command = _redoStack.Pop();
    command.Execute();
    _undoStack.Push(command);
}
}

// Usage:
var editor = new TextEditor();
var history = new HistoryManager();

history.Execute(new AppendTextCommand(editor, "Hello"));
history.Execute(new AppendTextCommand(editor, " World"));
Console.WriteLine(editor.Content); // Hello World

history.Undo();
Console.WriteLine(editor.Content); // Hello

history.Redo();
Console.WriteLine(editor.Content); // Hello World

```

Usage

- UI action binding, job queues, scheduled tasks, macro recording, undo/redo workflows,
- auditing and replay by persisting command history,
- remote or delayed execution boundaries,
- batch processing where commands are queued then executed by workers.

Examples

- desktop editor maps toolbar buttons to command objects and supports undo stack,

- background worker persists command payloads to queue and executes them later.

Edge Cases

- command payloads can become stale if underlying state changes before execution,
- undo logic is often harder than execute logic and easy to get wrong,
- high command volume can increase memory/storage pressure.

Best Practices

- design commands to be small, explicit, and serializable when needed,
- separate command validation from execution side effects,
- include correlation IDs/metadata for observability in distributed systems.

9.4 Strategy

Intent

Define a family of algorithms, encapsulate each one, and make them interchangeable.

Problem It Solves

Removing `if/else` algorithm switching from core flow.

Visualization

```
Context --> IStrategy
          |-- StrategyA
          |-- StrategyB
```

C# Example

```
public interface IShippingStrategy
{
    decimal Calculate(decimal packageWeightKg, decimal distanceKm);
}

public class StandardShippingStrategy : IShippingStrategy
{
    public decimal Calculate(decimal packageWeightKg, decimal distanceKm) =>
        5m + (packageWeightKg * 0.8m) + (distanceKm * 0.02m);
}

public class ExpressShippingStrategy : IShippingStrategy
{
    public decimal Calculate(decimal packageWeightKg, decimal distanceKm) =>
        15m + (packageWeightKg * 1.2m) + (distanceKm * 0.04m);
}

public class FreeShippingStrategy : IShippingStrategy
{
    public decimal Calculate(decimal packageWeightKg, decimal distanceKm) => 0m;
}

public class ShippingCalculator
{
    private readonly IShippingStrategy _strategy;

    public ShippingCalculator(IShippingStrategy strategy) => _strategy = strategy;

    public decimal GetCost(decimal weightKg, decimal distanceKm) =>
        _strategy.Calculate(weightKg, distanceKm);
}

// Usage:
var standard = new ShippingCalculator(new StandardShippingStrategy());
var express = new ShippingCalculator(new ExpressShippingStrategy());
var free = new ShippingCalculator(new FreeShippingStrategy());

Console.WriteLine(standard.GetCost(2.5m, 120m)); // normal checkout
Console.WriteLine(express.GetCost(2.5m, 120m)); // urgent delivery
Console.WriteLine(free.GetCost(2.5m, 120m)); // promo campaign
```

Usage

- pricing and discount engines,
- serialization/compression algorithm switching,
- policy-based behavior selected by user tier, region, or feature flags,
- shipping/payment/risk policies selected at runtime by business rules.

Examples

- checkout chooses discount strategy by campaign code (`NoDiscount` , `SeasonalDiscount` , `VipDiscount`),
- file service switches compression strategy (`Gzip` , `Brotli` , `None`) per endpoint requirements.

Edge Cases

- strategy selection can become scattered across code if not centralized,
- excessive tiny strategies can increase complexity without clear value,
- stateful strategies can cause shared-state bugs when reused.

Best Practices

- keep strategies stateless whenever possible,
- centralize selection logic (factory/resolver) outside business flow,
- use meaningful names that describe business intent, not technical mechanics.

9.5 State

Intent

Allow an object to alter its behavior when its internal state changes. The object appears to change its class.

Problem It Solves

Complex state-specific logic spread across big `switch` statements.

Strategy vs State (frequent confusion)

- **Strategy**: client chooses algorithm.
- **State**: object transitions itself between states.

Visualization

```
Context -> IState.Handle(context)
        |-- OpenState
        |-- ClosedState
```

C# Example

```
public class StateResult
{
    public bool Success { get; set; }
    public string Message { get; set; } = "";
    public string FromState { get; set; } = "";
    public string ToState { get; set; } = "";
}

public interface IOrderState
{
    StateResult Pay(OrderContext context);
    StateResult Ship(OrderContext context);
    StateResult Cancel(OrderContext context);
    string Name { get; }
}

public class NewOrderState : IOrderState
{
    public string Name => "New";

    public StateResult Pay(OrderContext context)
    {
        var next = new PaidOrderState();
        context.SetState(next);
        return new StateResult
        {
            Success = true,
            Message = "Payment accepted",
            FromState = Name,
            ToState = next.Name
        };
    }

    public StateResult Ship(OrderContext context) =>
        new StateResult
        {
            Success = false,
            Message = "Cannot ship before payment",
            FromState = Name,
            ToState = Name
        };
}
```

```

public StateResult Cancel(OrderContext context)
{
    var next = new CanceledOrderState();
    context.SetState(next);
    return new StateResult
    {
        Success = true,
        Message = "Order canceled",
        FromState = Name,
        ToState = next.Name
    };
}
}

```

```

public class PaidOrderState : IOrderState
{
    public string Name => "Paid";

    public StateResult Pay(OrderContext context) =>
        new StateResult
        {
            Success = false,
            Message = "Order is already paid",
            FromState = Name,
            ToState = Name
        };
}

```

```

public StateResult Ship(OrderContext context)
{
    var next = new ShippedOrderState();
    context.SetState(next);
    return new StateResult
    {
        Success = true,
        Message = "Order shipped",
        FromState = Name,
        ToState = next.Name
    };
}

```

```

public StateResult Cancel(OrderContext context)
{

```



```

        var next = new CanceledOrderState();
        context.SetState(next);
        return new StateResult
        {
            Success = true,
            Message = "Order canceled with refund",
            FromState = Name,
            ToState = next.Name
        };
    }
}

public class ShippedOrderState : IOrderState
{
    public string Name => "Shipped";

    public StateResult Pay(OrderContext context) =>
        new StateResult
        {
            Success = false,
            Message = "Already paid and shipped",
            FromState = Name,
            ToState = Name
        };

    public StateResult Ship(OrderContext context) =>
        new StateResult
        {
            Success = false,
            Message = "Order is already shipped",
            FromState = Name,
            ToState = Name
        };

    public StateResult Cancel(OrderContext context) =>
        new StateResult
        {
            Success = false,
            Message = "Cannot cancel after shipping",
            FromState = Name,
            ToState = Name
        };
}

```

```

public class CanceledOrderState : IOrderState
{
    public string Name => "Canceled";

    public StateResult Pay(OrderContext context) =>
        new StateResult
        {
            Success = false,
            Message = "Canceled order cannot be paid",
            FromState = Name,
            ToState = Name
        };

    public StateResult Ship(OrderContext context) =>
        new StateResult
        {
            Success = false,
            Message = "Canceled order cannot be shipped",
            FromState = Name,
            ToState = Name
        };

    public StateResult Cancel(OrderContext context) =>
        new StateResult
        {
            Success = false,
            Message = "Order is already canceled",
            FromState = Name,
            ToState = Name
        };
}

public class OrderContext
{
    private IOrderState _state = new NewOrderState();
    public string OrderId { get; }
    public List<string> TransitionHistory { get; } = new();

    public OrderContext(string orderId) => OrderId = orderId;

    public void SetState(IOrderState state) => _state = state;
}

```

```

public StateResult Pay() => Track(_state.Pay(this), "Pay");
public StateResult Ship() => Track(_state.Ship(this), "Ship");
public StateResult Cancel() => Track(_state.Cancel(this), "Cancel");
public string CurrentState => _state.Name;

private StateResult Track(StateResult result, string action)
{
    TransitionHistory.Add(
        $"{DateTime.UtcNow:HH:mm:ss} | {action} | {result.FromState} -> {result.ToS
    return result;
}
}

// Usage:
var order = new OrderContext("ORD-9001");
Console.WriteLine($"Initial: {order.CurrentState}");

var r1 = order.Ship();    // invalid transition
var r2 = order.Pay();     // New -> Paid
var r3 = order.Ship();    // Paid -> Shipped
var r4 = order.Cancel();  // blocked after shipping

Console.WriteLine($"{r1.Success} | {r1.Message}");
Console.WriteLine($"{r2.Success} | {r2.FromState} -> {r2.ToState}");
Console.WriteLine($"{r3.Success} | {r3.FromState} -> {r3.ToState}");
Console.WriteLine($"{r4.Success} | {r4.Message}");
Console.WriteLine($"Final: {order.CurrentState}");

foreach (var item in order.TransitionHistory)
    Console.WriteLine(item);

```

Usage

- workflow engines (draft/review/published),
- payment/order lifecycle transitions,
- protocol/session handling with explicit state transitions,
- business process guards where valid actions depend on current state.

Examples

- support ticket moves through New -> Assigned -> WaitingForCustomer -> Closed states with different behavior,

- payment state machine transitions Pending -> Authorized -> Captured and blocks invalid operations.

Edge Cases

- invalid transitions can appear if transitions are not guarded,
- state explosion occurs when too many fine-grained states are modeled,
- debugging becomes difficult if transitions are implicit or distributed.

Best Practices

- keep transition rules inside state objects, not scattered in context,
- log state transitions for auditability and troubleshooting,
- define a state diagram and enforce transition invariants with tests.

9.6 Chain of Responsibility

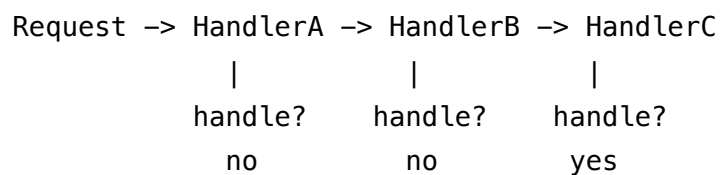
Intent

Avoid coupling sender of a request to its receiver by giving more than one object a chance to handle the request.

Problem It Solves

When request handling depends on ordered responsibility and each handler can pass forward.

Visualization



C# Example

```
public class ExpenseRequest
{
    public string Employee { get; set; } = "";
    public decimal Amount { get; set; }
    public string Purpose { get; set; } = "";
}

public abstract class ApprovalHandler
{
    protected ApprovalHandler? Next;

    public ApprovalHandler SetNext(ApprovalHandler next)
    {
        Next = next;
        return next;
    }

    public abstract void Handle(ExpenseRequest request);
}

public class TeamLeadApprovalHandler : ApprovalHandler
{
    public override void Handle(ExpenseRequest request)
    {
        if (request.Amount <= 300m)
            Console.WriteLine($"Team Lead approved {request.Amount} for {request.Employee}");
        else
            Next?.Handle(request);
    }
}

public class DepartmentManagerApprovalHandler : ApprovalHandler
{
    public override void Handle(ExpenseRequest request)
    {
        if (request.Amount <= 1500m)
            Console.WriteLine($"Department Manager approved {request.Amount} for {request.Employee}");
        else
            Next?.Handle(request);
    }
}
```

```

public class FinanceDirectorApprovalHandler : ApprovalHandler
{
    public override void Handle(ExpenseRequest request)
    {
        if (request.Amount <= 10000m)
            Console.WriteLine($"Finance Director approved {request.Amount} for {request}");
        else
            Console.WriteLine($"Rejected: {request.Amount} exceeds approval policy");
    }
}

// Usage:
var teamLead = new TeamLeadApprovalHandler();
var manager = new DepartmentManagerApprovalHandler();
var finance = new FinanceDirectorApprovalHandler();

teamLead.SetNext(manager).SetNext(finance);

teamLead.Handle(new ExpenseRequest { Employee = "Anna", Amount = 180m, Purpose = "Taxi" });
teamLead.Handle(new ExpenseRequest { Employee = "Aram", Amount = 900m, Purpose = "Conference" });
teamLead.Handle(new ExpenseRequest { Employee = "Narek", Amount = 7000m, Purpose = "Travel" });
teamLead.Handle(new ExpenseRequest { Employee = "Mariam", Amount = 18000m, Purpose = "Equipment" });

```

Usage

- validation pipelines,
- HTTP middleware-like request processing,
- support escalation chains and approval workflows,
- security/policy checks where each handler can approve, reject, or delegate.

Examples

- request validation chain: auth handler -> schema validator -> business rule validator,
- enterprise approval flow: team lead -> department manager -> finance controller,
- claim processing pipeline: fraud check -> policy coverage check -> payout approval.

Edge Cases

- request can be dropped silently if no terminal/default handler exists,
- handler order bugs can produce unexpected behavior,
- side effects in early handlers can conflict with later handlers.

Best Practices

- always define a terminal fallback handler or explicit "not handled" result,
- make chain order explicit and covered by tests,
- keep handlers focused and idempotent where possible.

10) When to Use Patterns (and When Not To)

Use a Pattern When

- the problem is recurring,
- team members recognize the pattern vocabulary,
- trade-offs are acceptable,
- it improves maintainability or extensibility measurably.

Avoid a Pattern When

- code is still simple and stable,
- pattern adds abstraction with no real need,
- you are introducing it "because architecture should look advanced."

Good architecture is not maximum patterns.

Good architecture is minimum complexity for required flexibility.

11) Quick Comparison Matrix

Pattern	Primary Goal	Typical Risk
Singleton	Single shared instance	Hidden global state
Factory Method	Delegate product creation	Too many tiny factor
Abstract Factory	Create compatible product families	Added complexity upf
Builder	Construct complex objects safely	Mutable intermediate
Adapter	Integrate incompatible interface	Extra indirection
Decorator	Add behavior dynamically	Deep wrapping chain
Composite	Uniform tree processing	Over-generalized com
Iterator	Encapsulate traversal	Custom iterator bugs
Observer	Decoupled event notifications	Notification storms
Command	Encapsulate requests	Command object proli
Strategy	Swap algorithms	Too many strategy cl
State	State-driven behavior transitions	State explosion
Chain of Responsibility	Flexible multi-step handling	Hard tracing of flow

12) Final Key Message

Design patterns are professional communication tools and design accelerators.
Learn their intent deeply, not just their UML shape.

The real senior-level skill is:

- seeing the real problem,
- choosing the simplest fitting pattern,
- and stopping before overengineering begins.

If you remember one sentence, let it be this:

"A pattern is useful when it makes future change easier than today."